DOCUMENTACIÓN DEL MVP

Klimate

Mapa interactivo y consulta de condiciones meteorológicas por ubicación.

Resultado actual

El usuario puede seleccionar un punto en el mapa o usar la ubicación de su dispositivo. La aplicación consulta Open-Meteo y muestra las condiciones actuales en un panel simple.

Tecnologías

React, TypeScript, Vite, Leaflet, OpenStreetMap y Open-Meteo.

Alcance del documento

Describe el funcionamiento visible del prototipo, la separación técnica de responsabilidades, los patrones implementados y una propuesta de evolución para completar el catálogo de patrones del proyecto.

Versión MVP 1.0

Proyecto académico de clima y recomendaciones por actividad.

Cómo funciona

El MVP trabaja con coordenadas. Todo el flujo se inicia con una ubicación y termina con datos normalizados, listos para que la interfaz los muestre.

1 Seleccionar ubicación

El usuario hace clic en el mapa o pulsa “Usar mi ubicación”. Ambas acciones producen latitud y longitud.

2 Actualizar el estado

useSelectedLocation guarda el punto elegido. React actualiza el marcador, el panel y activa la consulta.

3 Consultar condiciones

useClimate usa ClimateFacade. La fachada delega la consulta al adaptador de Open-Meteo.

4 Normalizar la respuesta

OpenMeteoAdapter convierte la respuesta externa en ClimateData: temperatura, sensación térmica, humedad, viento, precipitación y condición.

5 Mostrar resultado

WeatherPanel representa el estado de carga, posibles errores y los datos actuales. Si cambia el punto, se cancela la solicitud anterior.

Arquitectura actual

domain contiene modelos y contratos. **application** concentra las operaciones de negocio. **infrastructure** integra APIs externas. **presentation** contiene React, estado y componentes.

Capa	Responsabilidad	Ejemplos
Domain	Contratos y modelos independientes.	`Location, ClimateData, ClimateProvider.`
Application	Orquesta casos de uso.	`ClimateFacade, LocateUserCommand.`
Infrastructure	Conecta proveedores reales.	`OpenMeteoAdapter, BrowserLocationProvider.`
Presentation	Muestra e interactúa con el usuario.	`MapExplorer, WeatherPanel, hooks.`

Patrones implementados

Los patrones existentes separan los detalles de React y de los proveedores externos. No se añadieron solo para cumplir una lista: cada uno resuelve una dependencia concreta del MVP.

Observer

Qué resuelve: sincronizar la interfaz con la ubicación seleccionada.

Implementación: useSelectedLocation conserva el estado. MapExplorer publica el punto y App, useClimate, WeatherPanel y el marcador reaccionan a ese cambio.

Archivos:
`presentation/hooks/useSelectedLocation.ts`

Adapter

Qué resuelve: evitar que la aplicación dependa del formato de Open-Meteo.

Implementación: OpenMeteoAdapter pide los campos actuales y los traduce al modelo ClimateData. Además convierte los códigos WMO en descripciones legibles.

Archivos:
`infrastructure/open-meteo/OpenMeteoAdapter.ts`

Facade

Qué resuelve: exponer una única operación para pedir clima.

Implementación: ClimateFacade recibe un ClimateProvider y ofrece getCurrentClimate. La interfaz no conoce el proveedor de clima ni los parámetros HTTP.

Archivos:
`application/services/ClimateFacade.ts`

Command

Qué resuelve: encapsular una acción de usuario que puede variar por plataforma.

Implementación: LocateUserCommand solicita una ubicación mediante un contrato. BrowserLocationProvider adapta navigator.geolocation y devuelve Location.

Archivos:
`application/commands/LocateUserCommand.ts`

Comportamientos de calidad ya presentes

Las solicitudes de clima usan AbortController para cancelar una consulta anterior cuando el usuario elige otro punto. La ubicación del navegador tiene estados de carga y mensajes para permiso denegado, ubicación no disponible o tiempo agotado.

Patrones por implementar

La siguiente lista permite crecer hacia el requisito de diez patrones sin crear abstracciones artificiales. Cada patrón se propone para una capacidad real del producto.

Patrón	Cuándo aplicarlo	Posible implementación
Strategy	Cuando existan recomendaciones por actividad.	Una estrategia por actividad: correr, ciclismo, picnic o caminata. Cada una interpreta clima y contaminación con sus propios umbrales.
Factory Method	Al añadir más fuentes de datos.	Una fábrica crea ClimateProvider o AirQualityProvider según configuración: Open-Meteo, proveedor alterno o datos simulados para pruebas.
Repository	Al incorporar caché e historial.	ClimateRepository centraliza consultas, expiración de datos y almacenamiento local por coordenadas, sin exponer detalles a la interfaz.
Decorator	Al enriquecer una recomendación.	Decoradores agregan alertas por lluvia, calor, UV o contaminación a una recomendación base sin modificar la regla original.
Builder	Al crear un reporte compartible.	RecommendationReportBuilder arma un resumen con punto, actividad, clima, aire, alertas y sugerencias opcionales.
Singleton	Al configurar recursos compartidos.	Una sola instancia de caché, configuración de proveedores o cliente HTTP. Debe usarse solo si se necesita estado global controlado.

Criterio para añadir un patrón

Cada patrón debe responder a un cambio previsto, una variación de comportamiento o una responsabilidad que necesite aislamiento. Si no reduce acoplamiento ni mejora pruebas o extensibilidad, es mejor no añadirlo.

Próximo incremento recomendado

Añadir calidad del aire desde Open-Meteo y un Strategy por actividad. Esa evolución activa una necesidad concreta de reglas intercambiables y abre el camino a Decorator para las alertas.